

Top 25 Python String Questions

Part 1: Theory (Q1 - Q10)

Q1.

Strings in Python are:

- A) Mutable
- B) Immutable**
- C) Mutable only with +=
- D) Depends on Python version

Q2.

Which of these does NOT modify a string in place (returns a new string instead)?

- A) .upper()
- B) .replace()
- C) .strip()
- D) All string methods return new strings — none modify in place**

Q3.

Which is the most reliable way to check if two strings have the same VALUE (not identity)?

- A) s1 is s2
- B) s1 == s2**
- C) id(s1) == id(s2)
- D) hash(s1) is hash(s2)

Q4.

s.split() vs s.split(" ") — which statement is TRUE?

- A) Both behave identically in all cases
- B) split() collapses consecutive whitespace and ignores leading/trailing spaces; split(" ") treats every single space as a separate delimiter**
- C) split(" ") is just a faster alias for split()
- D) split() cannot handle tabs or newlines

Q5.

Why is repeated string concatenation with + inside a loop slower than "".join(list)?

- A) The + operator doesn't work on strings at all
- B) Each + creates a brand-new string object, making repeated concatenation roughly O(n²) overall, while join is O(n)**
- C) join() uses more memory but runs faster in theory only
- D) There is no real performance difference in modern Python

Q6.

What does string interning mean in Python?

- A) Strings are encrypted internally for security
- B) Certain strings (like short literals/identifiers) may be cached and reused, so identical values can share the same object in memory
- C) All strings are automatically converted to bytes internally
- D) Strings longer than 20 characters are split internally

Q7.

Regarding f-strings vs .format() vs % formatting, which is TRUE?

- A) All three have identical performance and are purely a style choice
- B) f-strings are evaluated at runtime and are generally the fastest and most readable of the three
- C) % formatting is deprecated and raises errors in Python 3
- D) .format() cannot access variables, only positional arguments

Q8.

Why can `s1 is s2` sometimes return True and sometimes False for two strings with identical content?

- A) It's random / non-deterministic behavior
- B) Python may intern short, simple string literals (reusing the same object), but strings built dynamically (concatenation, input, slicing, etc.) usually aren't guaranteed to be interned
- C) `is` always returns True for strings
- D) `is` always returns False for strings

Q9.

Which is TRUE about the `in` operator for substring checks, e.g. `"ab" in "abcabc"`?

- A) It only checks the first character and ignores the rest
- B) It returns True/False, internally scanning for the substring (roughly $O(n*m)$ worst case)
- C) It raises an error if the substring isn't found
- D) It only works with single characters, not substrings

Q10.

Why can you do `"hello"[0]` but not `"hello"[0] = "H"`?

- A) Indexing is a bug that Python happens to allow
- B) Reading by index works because strings support `__getitem__`, but assignment is blocked since strings don't support `__setitem__` (immutability)
- C) Only lowercase strings support indexing
- D) The `[0] =` syntax doesn't exist anywhere in Python

Part 2: Logic-Heavy Code (Q11 - Q15)

Q11.

```
s = "abacabadabacaba"
print(s.count("aba"))
print(s.find("aba", 5))
```

- A) 4 8
- B) 4 4
- C) 5 8
- D) 4 12

Q12.

```
s = "encyclopedia"
mid = len(s)//2
print(s[:mid], s[mid:])
print(sorted(s[:mid]) == sorted(s[mid:]))
```

- A) "encycl" "opedia" False
- B) "encycl" "opedia" True
- C) "encyc" "lopedia" False
- D) "encycl" "pedia" True

Q13.

```
s = "aabbccddeeffgg"
print(len(set(s)))
print(len(s) // len(set(s)))
```

- A) 7 2
- B) 14 2
- C) 7 7
- D) 8 1

Q14.

```
s = "level madam noon"
print([w for w in s.split() if w == w[::-1]])
print(all(w == w[::-1] for w in s.split()))
```

- A) ['level','madam','noon'] True
- B) ['level','madam','noon'] False
- C) ['level','noon'] False
- D) [] False

Q15.

```
s = "abcdefghijklmnopqrstuvwxyz"  
print(s[:5])  
print(s[4:5])
```

- A) "afkpuz" "ejoty"
- B) "afkpuz" "ejotyz"
- C) "afkpu" "ejoty"
- D) "afkps" "ejoty"

Part 3: Medium Difficulty Code (Q16 - Q25)

Q16.

```
s = "hello world"
print(s.replace("l", "L", 1))
```

- A) "heLlo world"
- B) "heLLo world"
- C) "hello worLd"
- D) "HeLlo world"

Q17.

```
s = "Data123Science456"
print("".join(c for c in s if not c.isdigit()))
```

- A) "DataScience"
- B) "123456"
- C) "Data Science"
- D) "DataScience123456"

Q18.

```
s = "abcabcabc"
print(s.index("c"))
print(s.rindex("c"))
```

- A) 2 8
- B) 2 6
- C) 0 8
- D) 2 9

Q19.

```
s = "Python Programming"
print(s.split()[-1][::-1])
```

- A) "gnimmargorP"
- B) "nohtyP"
- C) "Programming"
- D) "gnimmargorp"

Q20.

```
s = "AAABBBCCC"  
print(s.lower().count("b"))
```

A) 3

B) 0

C) 9

D) 1

Q21.

```
s = "hello,world,foo"  
print(len(s.split(",")))
```

A) 2

B) 3

C) 4

D) 1

Q22.

```
s = " trim me "  
print(repr(s.strip()))
```

A) 'trim me'

B) ' trim me '

C) 'trim me'

D) 'trim me '

Q23.

```
s = "abcdef"  
print(s[1:4] + s[-2:])
```

A) "bcdef"

B) "bcdde"

C) "bcdff"

D) "abcef"

Q24.

```
s = "MadHatter"  
print(s.lower().count("a"))
```

A) 2

B) 3

C) 1

D) 0

Q25.

```
s = "test string here"  
print(s.replace(" ", "_"))
```

A) "test_string_here"

B) "test string_here"

C) "test_string here"

D) "test-string-here"